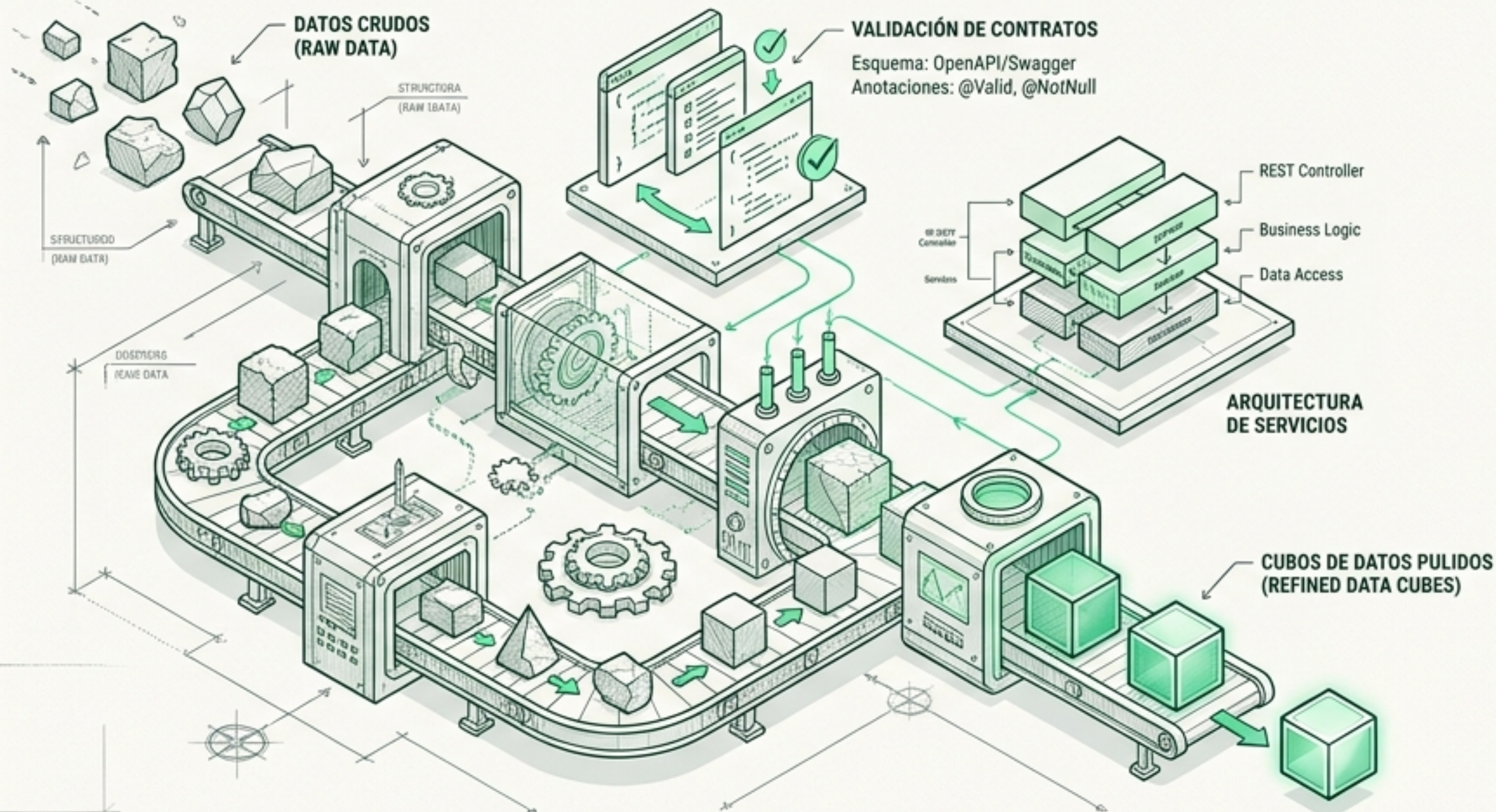
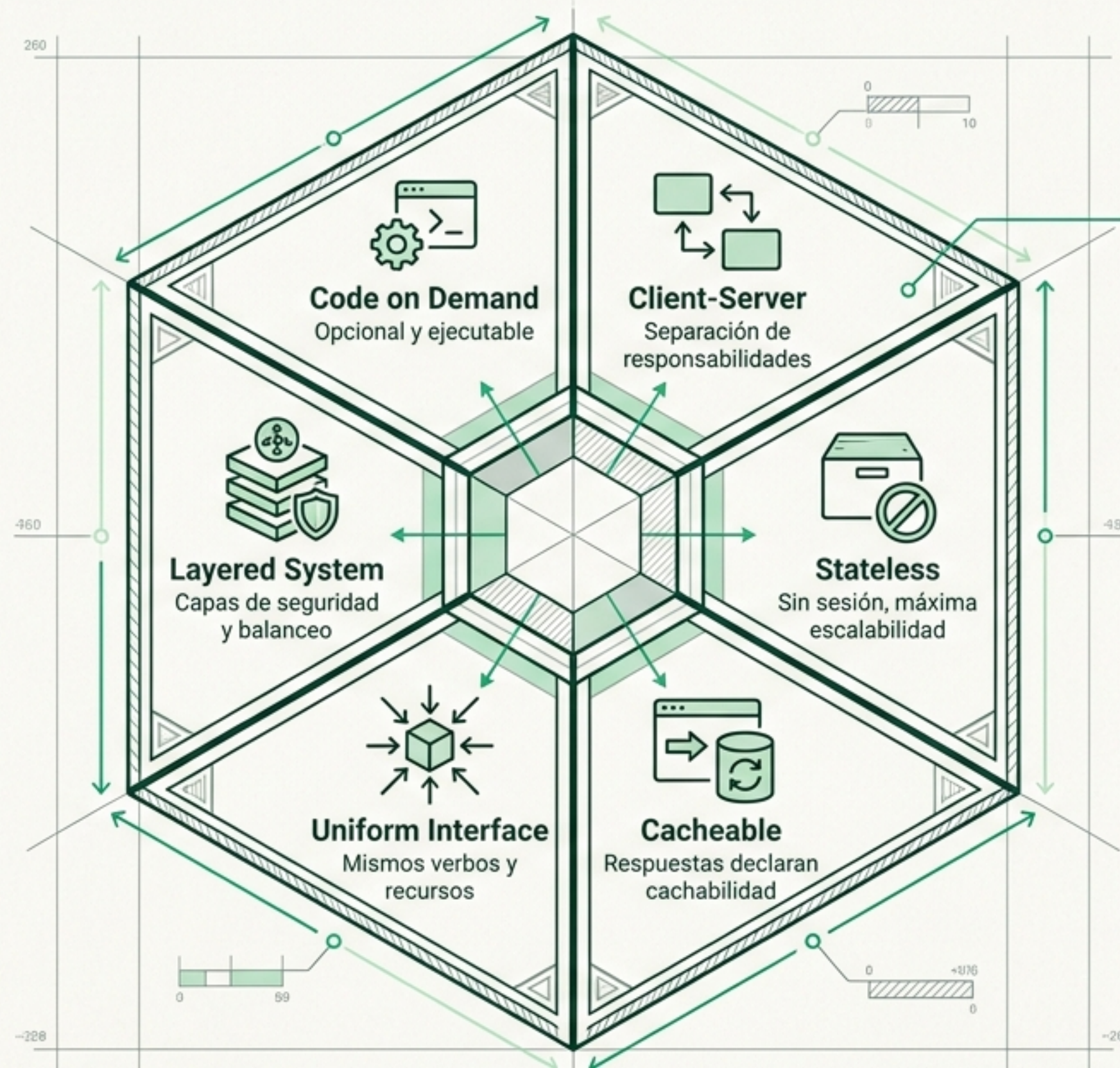


Construcción de APIs REST con Spring Boot

Módulo 5: Arquitectura, Validación y Contratos



Arquitectura RESTful: Los 6 Pilares de Roy Fielding



Nota de Diseño:

La mayoría de las APIs modernas son REST-like (HTTP + JSON). REST puro exige hipermedia (HATEOAS).

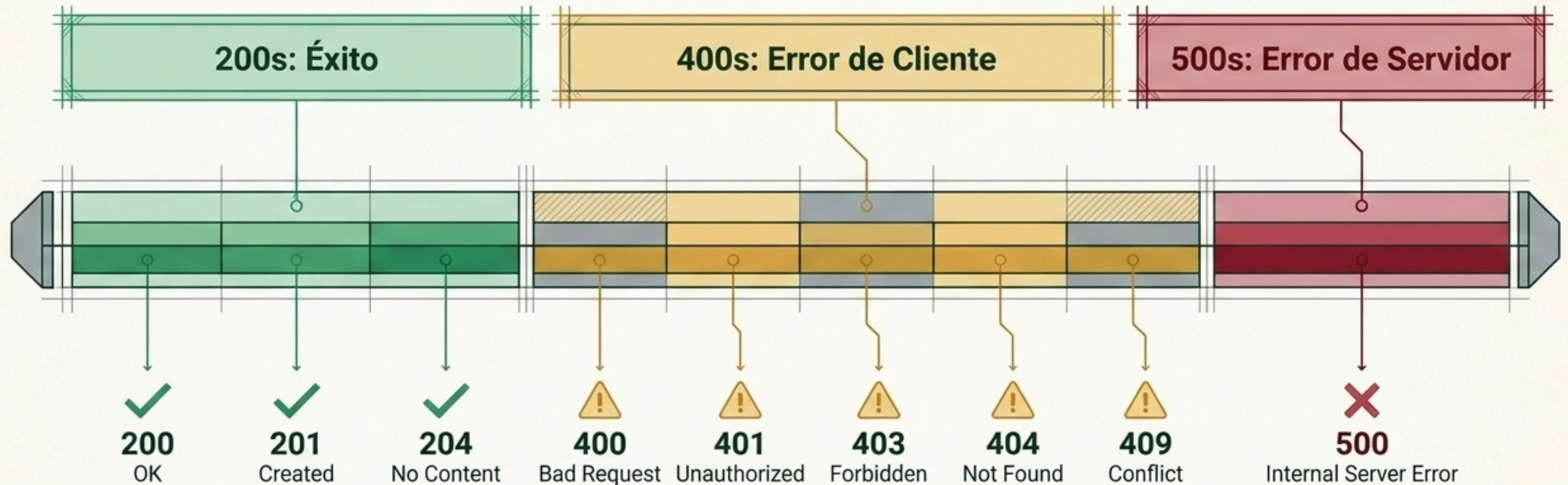
El Vocabulario HTTP

Idempotencia:

Es 100% seguro reintentar la petición si la red falla, sin efectos secundarios.

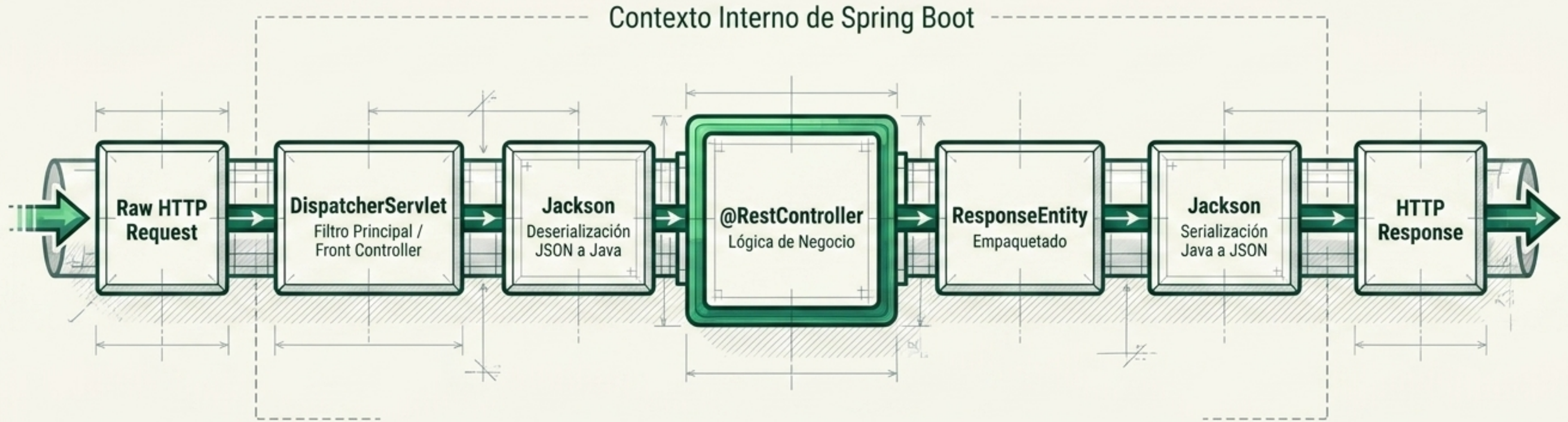
Verbo	Propósito	Idempotente	Requiere Body
GET	Leer recurso	✓	✗
POST	Crear recurso	✗	✓
PUT	Reemplazar recurso	✓	✓
PATCH	Actualizar parcial	✓	✓
DELETE	Eliminar recurso	✓	✗

El Espectro de Respuestas HTTP

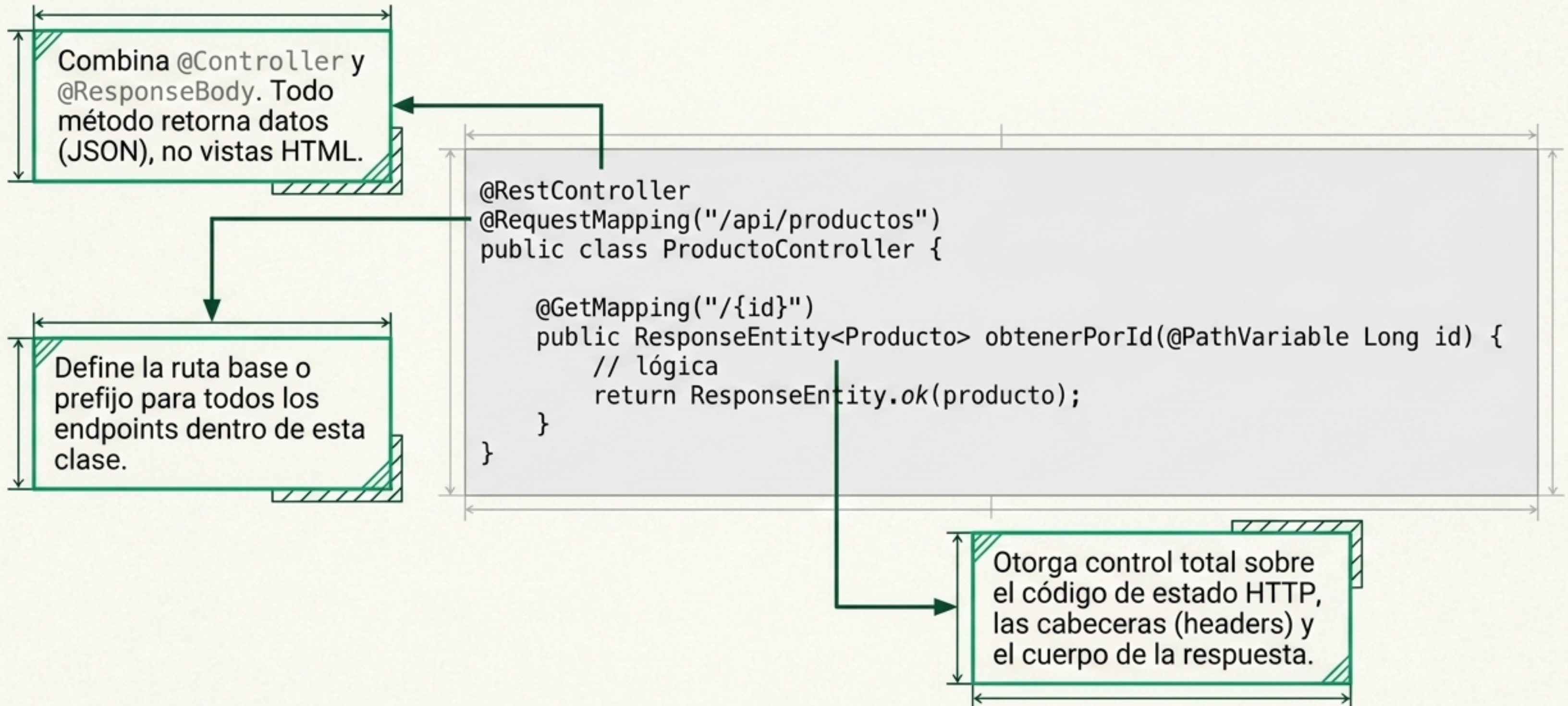


Regla de Oro: Tu API debe comunicarse usando este lenguaje universal. Nunca devuelvas un 200 OK con un mensaje de error dentro del JSON.

El Pipeline MVC de Spring



Anatomía de un Controlador REST

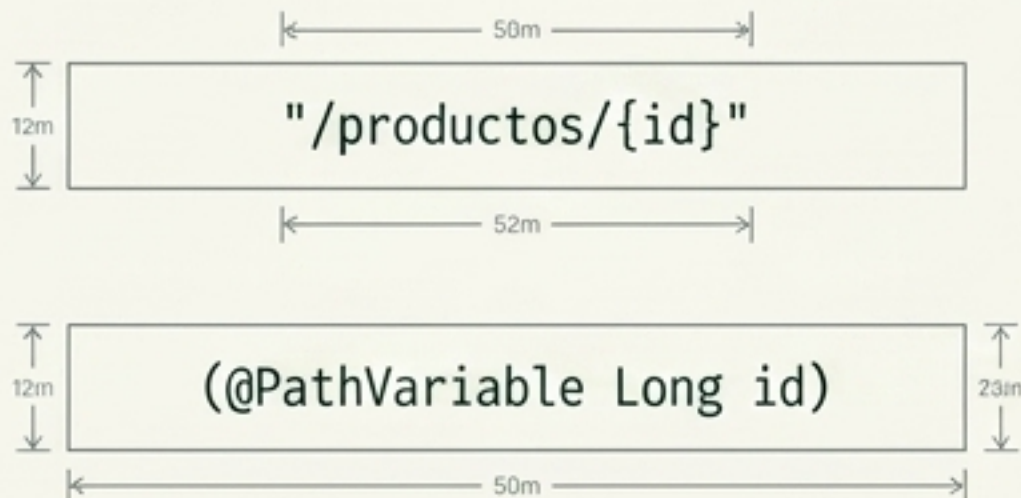


Las 3 Vías de Extracción de Datos



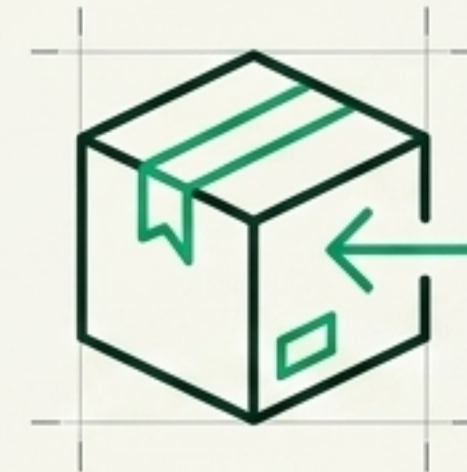
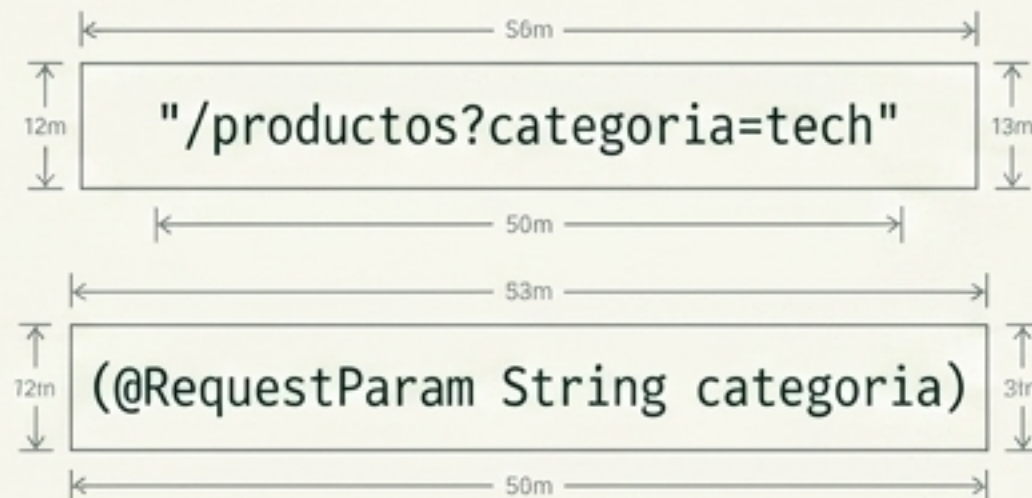
@PathVariable

Extraer identificadores incrustados directamente en la ruta de la URL.



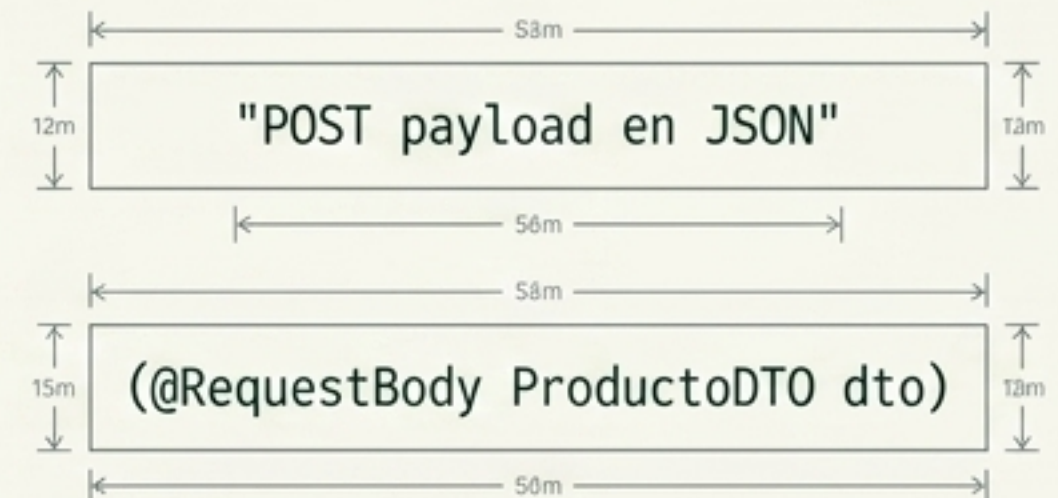
@RequestParam

Recibir filtros, parámetros de paginación u opciones al final de la URL.

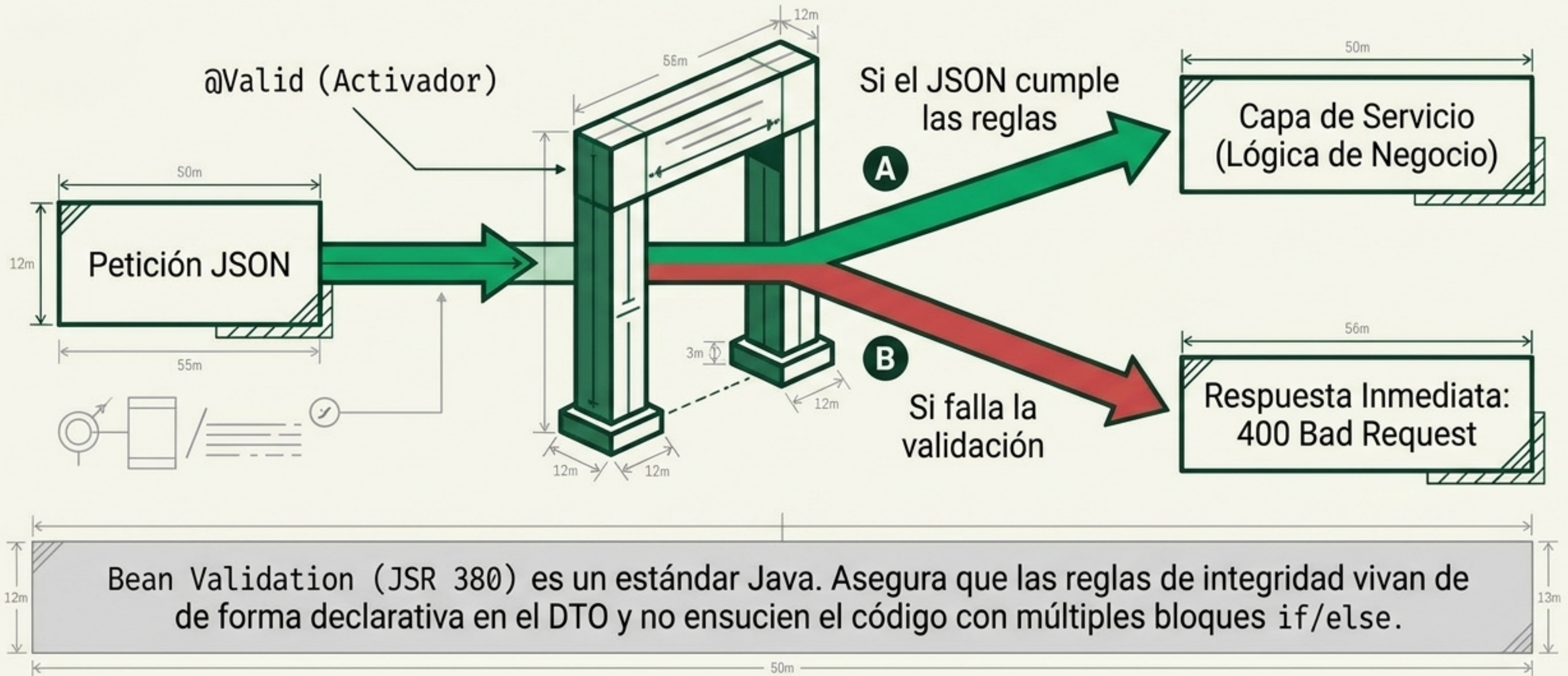


@RequestBody

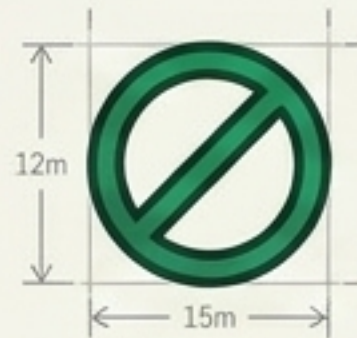
Deserializar payloads JSON complejos enviados en el cuerpo de la petición HTTP.



El Escudo de Bean Validation (JSR 380)

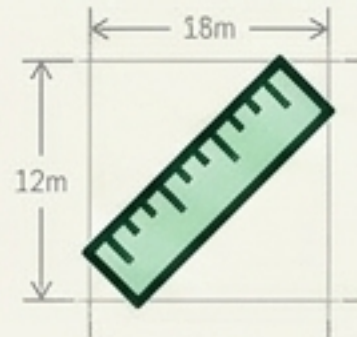


Diccionario de Restricciones (Referencia Rápida)



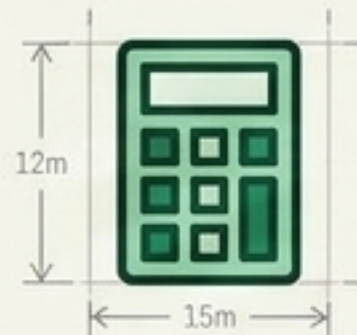
Control de Nulidad

- @NotNull (Rechaza nulos)
 - @NotBlank (Rechaza nulos, vacíos y solo espacios)
- Diagram dimensions: 28m height, 56m width.



Tamaño y Formato

- @Size(min, max) (Valida longitud de strings/listas)
 - @Email (Formato de correo)
 - @Pattern(regex) (Expresiones regulares)
- Diagram dimensions: 15m height, 56m width.

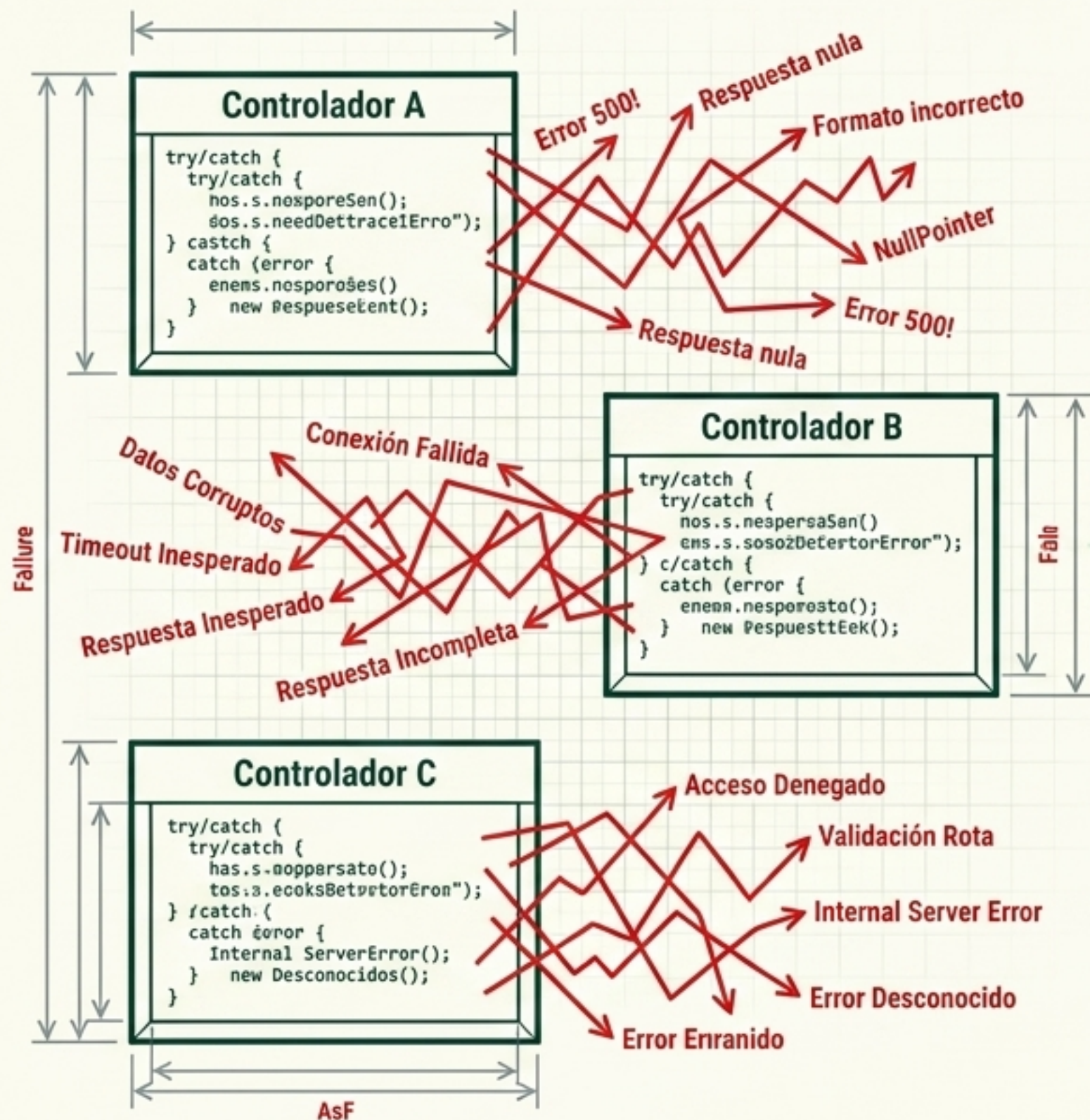


Límites Numéricos

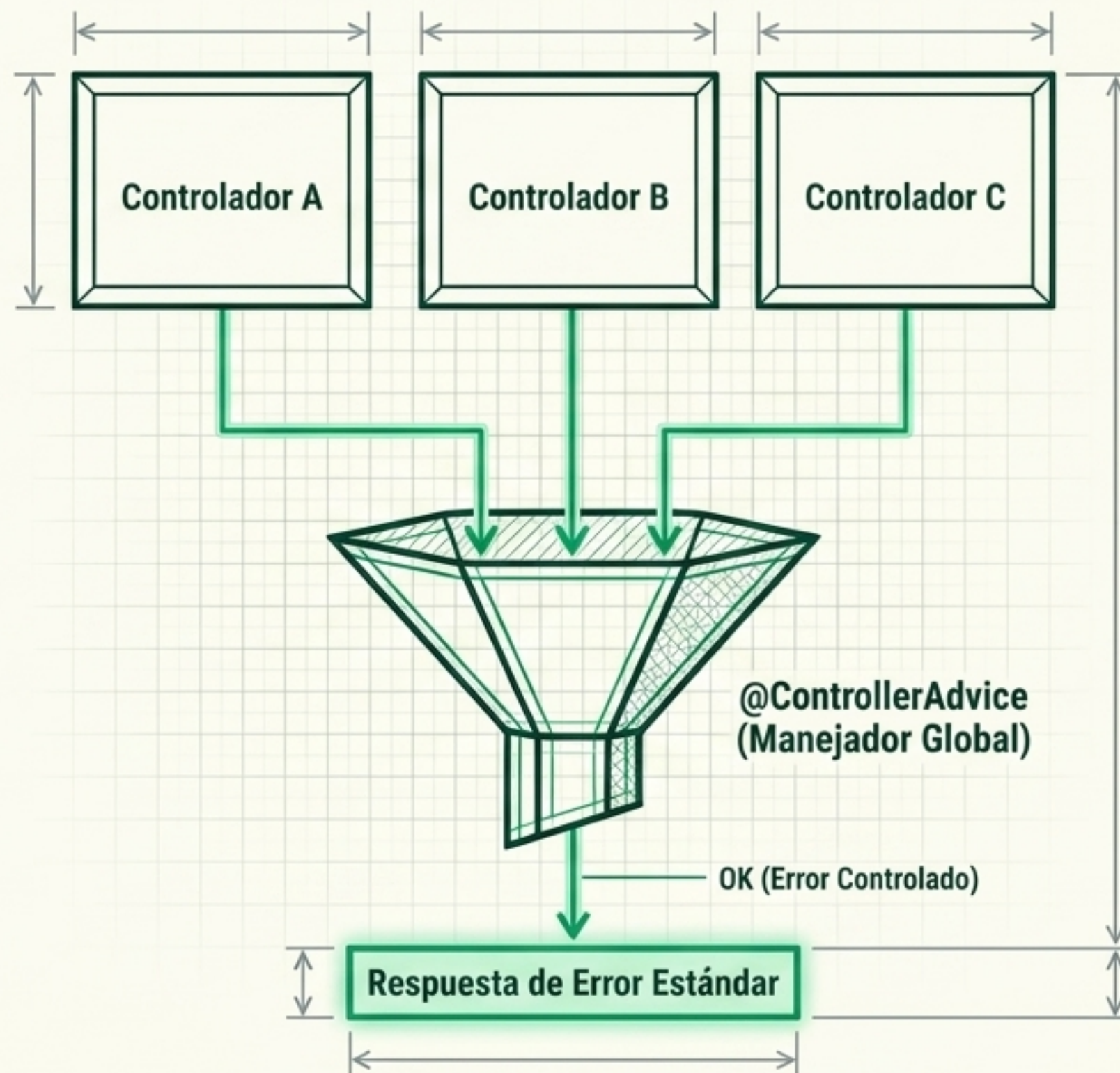
- @Min / @Max (Valores absolutos)
 - @Positive / @PositiveOrZero (Valores mayores a cero)
- Diagram dimensions: 12m height, 50m width.

Manejo de Errores: El Caos vs. El Orden

Antes: El Caos



Después: El Orden con Spring



Estandarización de Errores (RFC 9457)

```
{
```

```
  "type": "/errores/producto-no-encontrado",
```

```
  "title": "Recurso no encontrado",
```

```
  "status": 404,
```

```
  "detail": "El producto con ID 8 no existe  
en la base de datos.",
```

```
  "instance": "/api/productos/8"
```

```
}
```

URI que categoriza el error
(ideal para documentación).

Mensaje corto y legible
por humanos.

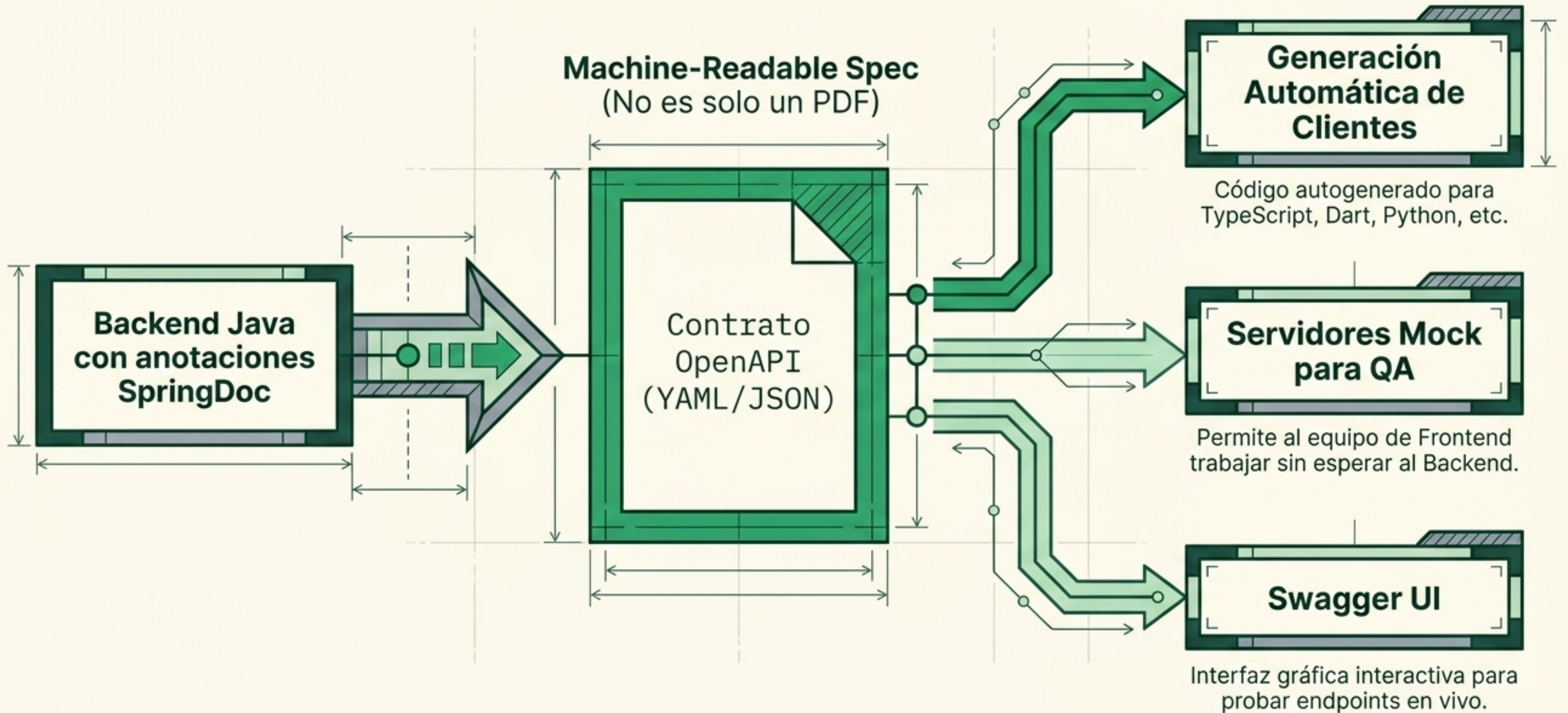
El código HTTP exacto de
la respuesta.

Explicación granular y
específica de esta falla.

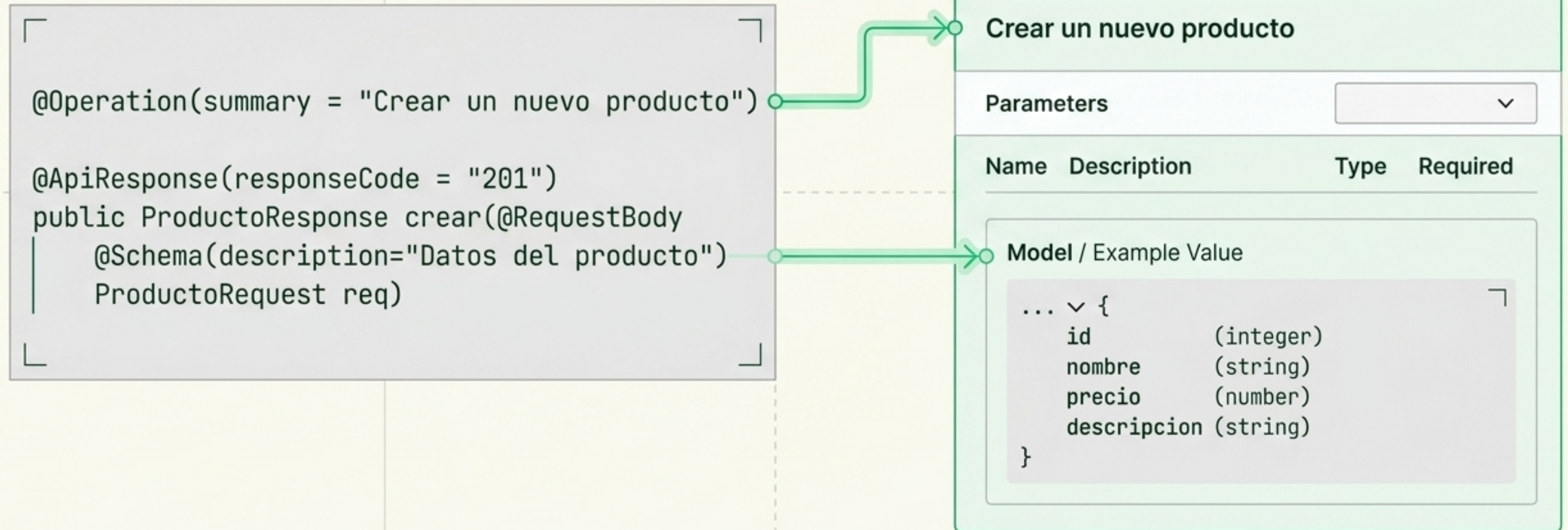
La URI del recurso exacto
que provocó el error.

Beneficio: Previene que los clientes (Web/Móvil) hagan parseo de strings (text-matching) para adivinar el error. Ofrece un código máquina consistente.

OpenAPI: El Contrato de Software

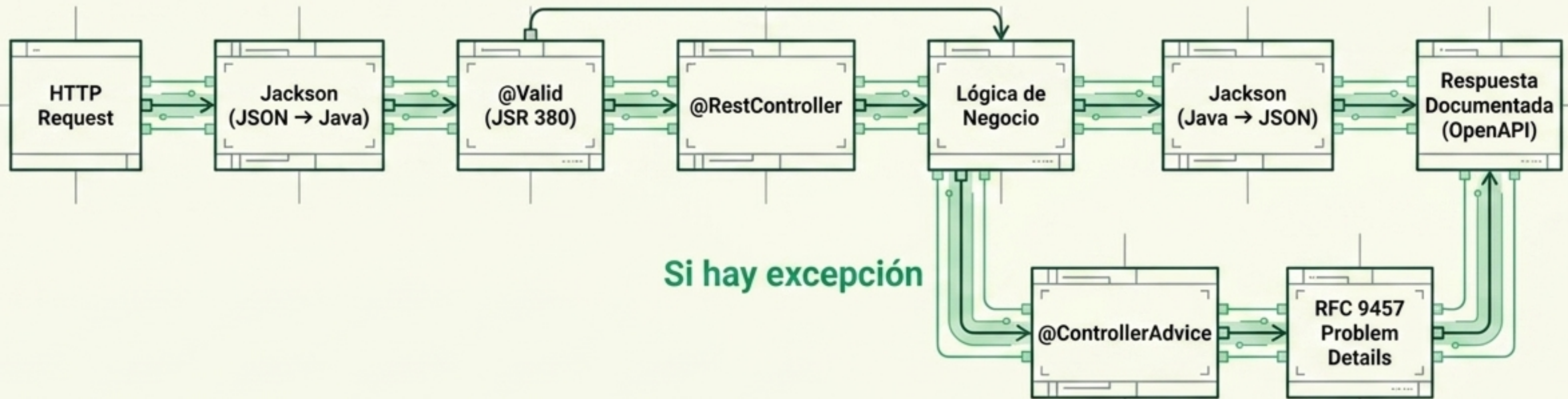


Magia Documental: Código ↔ Swagger



SpringDoc traduce en tiempo real: el código es la única fuente de la verdad.

La Síntesis: El Viaje Completo de un Request



Resumen Ejecutivo y Siguiente Paso

1. Estado y Vocabulario

REST delega el contexto al cliente. Las acciones se definen usando verbos HTTP estandarizados y Status Codes universales.

2. Barreras Estrictas

Bean Validation protege el dominio evaluando los DTOs antes de que la información corrupta toque la lógica de negocio.

3. Contratos Sincronizados

Swagger/OpenAPI y el manejo global de errores aseguran que el servidor y el cliente hablen el mismo idioma sin ambigüedades.

Siguiente Paso: Módulo 6 — Acceso a Datos
(Spring Data JPA y Flyway)